



Adventist University of Central Africa

P.O. Box 2461 Kigali, Rwanda | www.auca.ac.rw | info@auca.ac.rw

Department of Information Technology

Course Code and Name: INSY 8311 – Database Development with PL/SQL

Assignment 1: SQL Mastery

Deadline: Sunday, 20 September 2026, 11:59 PM

1. Purpose

Before we move into PL/SQL (stored procedures, functions, triggers, packages), you must demonstrate solid command of core SQL: querying, joining, aggregating, filtering, and manipulating relational data. This assignment uses the same schema you already set up in your XEPDB1 database as the **lecturer** user: **STUDENTS**, **COURSES**, **ENROLLMENTS**, **EMPLOYEES**, and **DEPARTMENTS**.

Every task must be solved with plain SQL (SELECT, INSERT, UPDATE, DELETE, CREATE VIEW, CREATE INDEX, DDL constraints, etc.). This is intentional: if you reach for a loop or a cursor to solve any of these, you are overcomplicating it; go back to a set-based SQL approach.

2. Submission Rules: Read Before You Start (Git Workflow)

Follow this exactly. Any deviation results in disqualification of your submission.

1. **Clone** the class repository, do not fork it, do not create your own new repository for this assignment.
2. **Create your own branch** dedicated to this assignment. Use the naming convention: **student_id_firstname_lastname_assignment1** (e.g. 20657_elie_kayitare_assignment1)
3. Do all your work on **that branch only**.
4. **Commit** your work with clear, incremental commit messages (not one giant "final" commit).
5. **Push your branch** to the remote repository.
6. **Do NOT push to main** and do NOT open a pull request that merges into main yourself. Submission is your pushed branch, nothing else.
7. Submitting via your own repository, a zip file, email, or any channel other than a pushed branch on the class repo will **not be graded**.

Disqualifying actions: pushing to main, creating a personal/separate repository for this work, or submitting outside your dedicated branch.

What to push

A single, dedicated folder for this assignment inside your branch:

1. Assignment1 folder with .sql file, readme.md and pdf report. Both .sql and pdf report should also have the same naming convention as your branch.

Each query in the .sql file must be preceded by a comment explaining the codes you have used.

The README.md report must be written by you. It must reflect your own testing process for your own queries. Copying another student's report, or generating it with an AI tool instead of describing your own verification steps, will be treated as an academic integrity violation.

3. Ground Rules

- ✓ Every query must actually run without error against the schema from the install/config guide (the one with 50 students, 15 courses, ~90 enrollments, 40 employees, 6 departments).
- ✓ Where a task asks for a specific output (e.g. "list names," "show the count"), your SELECT must return exactly those columns, sensibly aliased, not the whole row unless asked.
- ✓ Do not hardcode values you could otherwise derive (e.g. don't hardcode student_id = 1001 when the task says "the student with the highest GPA" find it with SQL).
- ✓ Where a task says "using a join" or "using a subquery," you must use that technique. Solving it a different way earns partial credit at best.
- ✓ Comment your SQL. A correct query with zero explanation of intent loses style points.

4. Tasks (100 points)

Section A: Filtering, Sorting, and Expressions (15 pts)

A1 (3 pts). List the first name, last name, and email of all students with a GPA of 3.5 or higher, ordered from highest to lowest GPA.

A2 (3 pts). List all courses in the "Computer Science" department that carry 4 credits, showing course name, course code, and instructor.

A3 (4 pts). Using CASE, produce a report of every student's name and a new column standing that shows 'Honors' for GPA ≥ 3.7 , 'Good' for GPA between 3.0 and 3.69, and 'Probation' for anything below 3.0.

A4 (5 pts). List all students whose email domain is email.com and who enrolled on or after 1-JAN-2024, showing name and enrollment date, sorted by enrollment date. (Use enrollment_date from STUDENTS, not ENROLLMENTS.)

Section B: Aggregation and Grouping (20 pts)

B1 (4 pts). For each department, show the department name and the average salary of its employees, rounded to 2 decimal places. Order by average salary descending.

B2 (4 pts). Show each course's course name and how many students are enrolled in it (count of rows in ENROLLMENTS per course_id), including courses with zero enrollments. Order by enrollment count descending.

B3 (4 pts). Using HAVING, list only the courses that have more than 5 enrollments.

B4 (4 pts). For each student, show their full name and their average score across all their enrollments, but only include students who have at least one enrollment with a non-NULL score. Round the average to 1 decimal place.

B5 (4 pts). Find the highest, lowest, and average GPA across all students in a single query, each in its own labeled column.

Section C: Joins (25 pts)

C1 (5 pts). List each enrollment as: student full name, course name, and grade. For all enrollments, using an inner join across the three relevant tables.

C2 (5 pts). List every course together with the department's budget for the department that offers it (match on courses.department to departments.department_name). Include courses even if no matching department budget is found.

C3 (5 pts). Using a self-join on EMPLOYEES, list every employee's name next to their manager's name. For employees with no manager, show 'No Manager' instead of NULL.

C4 (5 pts). List all students who have no enrollments at all (use a JOIN and filter for the missing side; do not use a subquery here).

C5 (5pts). List every department along with the name of its manager (join DEPARTMENTS.manager_id to EMPLOYEES.employee_id), and the number of employees in that department (from EMPLOYEES.department matching by name). One row per department.

Section D: Subqueries (20 pts)

D1 (5 pts). Using a subquery (not a join), list the names of students who are enrolled in "PL/SQL Programming" (course code CS202).

D2 (5 pts). Using a correlated subquery, list every course whose average enrollment score is above the overall average score across all enrollments.

D3 (5 pts). Find the employee(s) with the highest salary in their own department, using a correlated subquery (i.e., the employee's salary equals the max salary among employees in the same department).

D4 (5 pts). Using IN with a subquery, list the names of courses that have never had a single enrollment.

Section E: Data Definition & Manipulation (15 pts)

E1 (4 pts). Add a new column advisor_id NUMBER to the STUDENTS table, referencing employees(employee_id). Write the ALTER TABLE statement (do not just describe it).

E2 (4 pts). Write an UPDATE statement that raises the salary of every employee in the "Engineering" department by 5%.

E3 (3 pts). Write a DELETE statement that removes all enrollments with status = 'Active' and a grade of 'C' and a score below 73. Before writing the DELETE, write the equivalent SELECT you used to confirm which rows would be affected (include both in your submission).

E4 (4 pts). Create a view called honor_roll that shows the full name, GPA, and enrollment date of every student with a GPA of 3.7 or above, ordered by GPA descending. Then write the single SELECT * FROM honor_roll query that would be used to check it.

Section F: Bonus (up to 5 pts extra credit)

F1 (bonus pts). Using a window function (RANK() or DENSE_RANK()), rank students by GPA within their own enrollment cohort (group by enrollment_date, treating each distinct date as a cohort). Show student name, enrollment date, GPA, and rank.

A perfect SQL answer submitted outside the correct branch, or pushed to main, receives **0** for the assignment regardless of query quality.

5. Academic Integrity

You may discuss approaches with classmates, but the SQL you submit must be your own work, written and tested by you against your own database instance. Identical, unexplainable query files across two students will be treated as an academic integrity violation for both parties.

This includes the README.md and pdf report. It must be written in your own words, describing what you actually did and observed. A report that is copied, templated, or produced by an AI tool rather than reflecting your own verification work will be treated the same as copied SQL; an academic integrity violation.